

Sistemas embebidos

Instrucciones de control y llamadas a subrutina

μ C ATmega328P

Dr. Rodrigo Vázquez López

Universidad Autónoma de la Ciudad de México
Plantel San Lorenzo Tezonco

Semestre 2025-II

UACM

Universidad Autónoma
de la Ciudad de México

NADA HUMANO ME ES AJENO

Control de Flujo en Microcontroladores

Las **instrucciones de salto** permiten alterar **el flujo secuencial** de ejecución del programa, implementando **estructuras de control** como bucles, condicionales y llamadas a subrutinas.

Tipos de control de flujo:

- **Salto Incondicionales:** Siempre se ejecutan
- **Salto Condicionales:** Dependen del **estado** de las **banderas**
- **Llamadas a Subrutinas:** Permiten **reutilizar** código
- **Salto por Bits:** Basados en el **estado de bits** individuales
- **Interrupciones:** Control de flujo por **eventos externos**

Importante

El **Program Counter (PC)** almacena la **dirección** de la **siguiente instrucción** a ejecutar

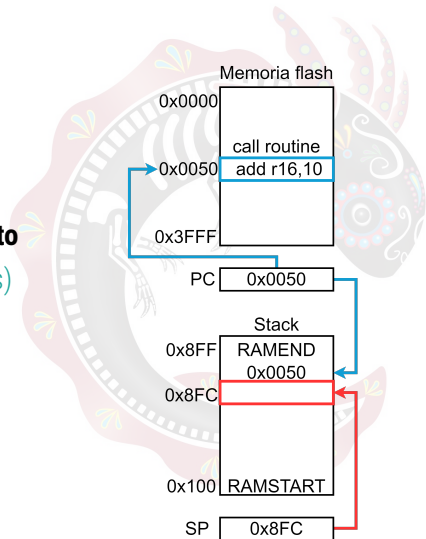
Program Counter (PC) y Stack

Program Counter (PC):

- Registro de **14 bits**
- Apunta a la **siguiente instrucción**
- Se **incrementa** automáticamente
- Modificado por **instrucciones de salto**
- Rango: 0x0000 - 0x3FFF (**16K words**)

Stack (Pila):

- Almacena **direcciones** de retorno
- Crece **hacia abajo** en **SRAM**
- Stack Pointer (SP) de **16 bits**
- Esencial para **CALL/RET**



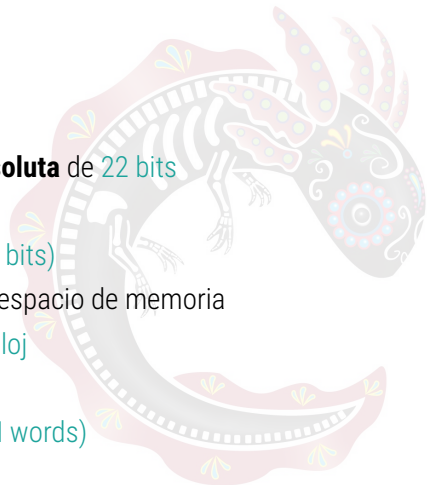
Instrucción JMP - Jump

JMP *k*

Salto incondicional a una **dirección absoluta** de 22 bits

Características:

- Ocupa **2 palabras** de programa (32 bits)
- Puede saltar a **cualquier parte** del espacio de memoria
- Tiempo de ejecución: 3 ciclos de reloj
- **No afecta banderas** del SREG
- Rango: 0x000000 - 0x3FFFFFF (4M words)



Instrucción JMP - Jump

Ejemplos:

```
1  JMP  main                ; Saltar a etiqueta 'main'
2  JMP  0x0100              ; Saltar a direccion absoluta
3  JMP  error_handler       ; Saltar a manejador de error
4
5  ;Codigo del programa principal
6  main:
7      LDI  r16, 0xFF
8      OUT  PORTB, r16
9      JMP  main            ; Bucle infinito
```

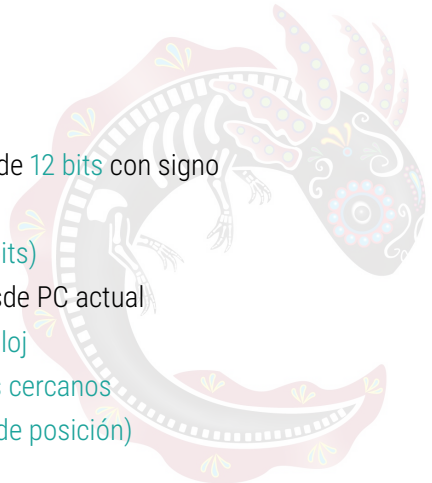
Instrucción RJMP - Relative Jump

RJMP k

Salto relativo incondicional con **offset** de 12 bits con signo

Características:

- Ocupa **1 palabra** de programa (16 bits)
- Rango: -2048 a +2047 palabras desde PC actual
- Tiempo de ejecución: 2 ciclos de reloj
- **Más eficiente** que **JMP** para saltos cercanos
- Código *relocatable* (independiente de posición)



Instrucción RJMP - Relative Jump

Ejemplos:

```
1 RJMP inicio           ; Salto relativo a 'inicio'
2 RJMP .+5              ; Saltar 5 instrucciones adelante
3 RJMP .-10             ; Saltar 10 instrucciones atras
4
5 bucle:
6     INC r16            ; Incrementar contador
7     CPI r16, 100       ; Llego a 100?
8     BRNE continuar     ; Si no, continuar
9     RJMP bucle         ; Si si, reiniciar bucle
10
11 continuar:
12 ; Continuar con el programa
```

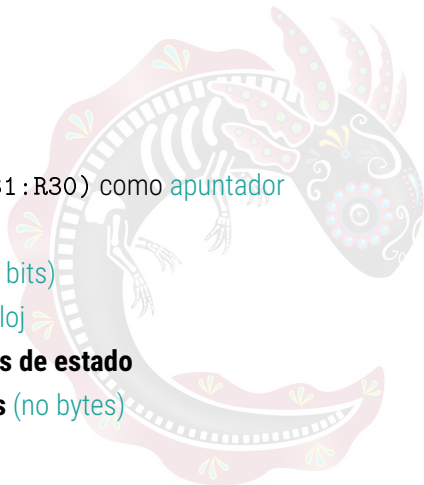
Instrucción IJMP - Indirect Jump

IJMP

Salto indirecto usando el **registro Z** (R31 : R30) como **apuntador**

Características:

- Dirección de salto en **registro Z** (16 bits)
- Tiempo de ejecución: 2 ciclos de reloj
- Útil para **tablas de salto** y **máquinas de estado**
- La dirección debe estar en **palabras** (no bytes)



Instrucción IJMP - Indirect Jump

Ejemplo - Tabla de salto:

```

1      LDI r16, 2          ; Seleccionar funcion 2
2      RCALL calcular_direccion
3      IJMP                ; Saltar a la funcion seleccionada
4
5  calcular_direccion:
6      LDI ZL, low(tabla_funciones*2) ; Direccion base
7      LDI ZH, high(tabla_funciones*2)
8      ADD ZL, r16          ; Sumar offset
9      ADC ZH, ZH           ; Propagar carry
10     RET
11
12  tabla_funciones:
13     RJMP funcion0        ; Funcion 0
14     RJMP funcion1        ; Funcion 1
15     RJMP funcion2        ; Funcion 2

```

Instrucción CALL - Call Subroutine

CALL k

Llamada absoluta a **subrutina** con dirección de 22 bits

Funcionamiento:

- 1 Guarda **PC+1** en el **stack** (dirección de retorno)
- 2 Salta a la **dirección especificada**
- 3 La **subrutina** termina con **RET**

Características:

- Ocupa **2 palabras** de programa
- Tiempo de ejecución: 4 ciclos de reloj
- Puede llamar a **cualquier parte** del programa
- Permite **anidamiento** de llamadas



Instrucción CALL - Call Subroutine

Ejemplo:

```
1 main:
2     LDI r16, 100           ; Cargar parametro
3     CALL delay_ms         ; Llamar subrutina de delay
4     ; Continúa aquí despues del RET
5     OUT PORTB, r16
6
7 delay_ms:                 ; Subrutina de delay
8     ; Codigo de delay usando r16
9     RET                   ; Retornar al llamador
```

Instrucción RCALL - Relative Call

RCALL k

Llamada relativa a subrutina con **offset** de 12 bits con signo

Características:

- Ocupa **1 palabra** de programa
- Rango: -2048 a +2047 palabras
- Tiempo de ejecución: 3 ciclos de reloj
- **Más eficiente** que **CALL** para subrutinas cercanas
- Código relocatable



Instrucción RCALL - Relative Call

Ejemplo - Sistema de menú:

```
1 menu_principal:
2     RCALL leer_teclea      ; Leer entrada del usuario
3     CPI r16, '1'          ; Opcion 1?
4     BREQ opcion1
5     CPI r16, '2'          ; Opcion 2?
6     BREQ opcion2
7     RJMP menu_principal   ; Repetir si opcion invalida
8
9 opcion1:
10    RCALL procesar_opcion1
11    RJMP menu_principal
12
13 opcion2:
14    RCALL procesar_opcion2
15    RJMP menu_principal
```

Instrucción ICALL - Indirect Call

ICALL

Llamada indirecta usando el **registro Z** como **apuntador**

Características:

- Dirección de llamada en **registro Z**
- Tiempo de ejecución: **3 ciclos de reloj**
- Útil para implementar **apuntadores a función**
- Permite **polimorfismo** en **sistemas embebidos**



Instrucción ICALL - Indirect Call

Ejemplo - Tabla de funciones:

```

1  LDI r16, 1 ; Seleccionar funcion matematica
2  RCALL ejecutar_funcion
3
4  ejecutar_funcion:
5  ; Calcular direccion de funcion
6  LDI ZL, low(tabla_matematicas*2)
7  LDI ZH, high(tabla_matematicas*2)
8  LSL r16 ; Multiplicar por 2 (tamano de entrada)
9  ADD ZL, r16 ; Sumar offset
10 ADC ZH, ZH ; Propagar carry
11
12 LPM r30, Z+ ; Cargar direccion baja
13 LPM r31, Z ; Cargar direccion alta
14 ICALL ; Llamar funcion
15 RET
16
17 tabla_matematicas:
18 .dw sumar*2, restar*2, multiplicar*2, dividir*2

```

Instrucciones RET y RETI

RET - Return from Subroutine

Retorna de una **subrutina** recuperando **PC** del **stack**

Operación RET:

- $PC \leftarrow \text{Stack}$ (recuperar dirección de retorno)
- $SP \leftarrow SP + 2$ (liberar espacio en stack)
- Tiempo de ejecución: 4 ciclos de reloj

RETI - Return from Interrupt

Retorna de **interrupción** y habilita **interrupciones globales**

Operación RETI:

- $PC \leftarrow \text{Stack}$ (como RET)
- $I \leftarrow 1$ (habilitar interrupciones globales)
- Tiempo de ejecución: 4 ciclos de reloj



Instrucciones RET y RETI

Ejemplos:

```
1 mi_subrutina:
2 ; Codigo de la subrutina
3     RET                ; Retorno normal
4
5 isr_timer:          ; Rutina de interrupcion
6     ; Guardar contexto
7     PUSH r16
8     ; Procesar interrupcion
9     POP r16          ; Restaurar contexto
10    RETI              ; Retorno de interrupcion
```

Saltos Basados en Banderas del SREG

Bit	Bandera	Descripción	Set	Clear
0	C	Carry	BRCS	BRCC
1	Z	Zero	BREQ	BRNE
2	N	Negative	BRMI	BRPL
3	V	Overflow	BRVS	BRVC
4	S	Sign	BRLT	BRGE
5	H	Half Carry	BRHS	BRHC
6	T	Transfer	BRTS	BRTC
7	I	Interrupt	BRIE	BRID

Características comunes:

- Rango: -64 a +63 palabras
- Tiempo de ejecución: 1 ciclo si **no salta**, 2 ciclos si **salta**

Saltos Basados en Banderas del SREG

Instrucciones BRBS y BRBC

Saltos condicionales basados en el **estado de banderas** individuales

BRBS s, k: Salta si el **bit s** del **SREG** está en **1**

BRBC s, k: Salta si el **bit s** del **SREG** está en **0**

Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
	T	H	S	V	N	Z	C

Saltos por Estado de Zero y Carry

BREQ/BRNE - Branch if Equal/Not Equal:

```

1 CPI r16, 100           ; Comparar con 100
2 BREQ es_cien           ; Saltar si es igual (Z=1)
3 BRNE no_es_cien        ; Saltar si no es igual (Z=0)

```

BRCS/BRCC - Branch if Carry Set/Clear:

```

1 ADD r16, r17           ; Sumar registros
2 BRCS overflow          ; Saltar si hay overflow (C=1)
3 BRCC sin_overflow      ; Saltar si no hay overflow (C=0)
4
5 ; Tambien se usan como:
6 ; BRLO (Branch if Lower) = BRCS
7 ; BRSH (Branch if Same or Higher) = BRCC
8 CPI r16, 50
9 BRLO menor_50          ; Saltar si r16 < 50 (sin signo)
10 BRSH mayor_igual_50   ; Saltar si r16 >= 50 (sin signo)

```

Saltos por Signo y Comparación

Comparaciones con signo:

```
1 CPI r16, 0           ; Comparar con 0
2 BRMI negativo        ; Saltar si negativo (N=1)
3 BRPL positivo        ; Saltar si positivo (N=0)
4
5 ; Para comparaciones con signo usar banderas S y V
6 CP r16, r17          ; Comparar r16 con r17
7 BRLT menor           ; Saltar si r16 < r17 (con signo)
8 BRGE mayor_igual     ; Saltar si r16 >= r17 (con signo)
```

Saltos por Signo y Comparación

Ejemplo - Bucle contador:

```
1      LDI r16, 10                ; Inicializar contador
2
3  bucle_contador:
4      ; Codigo del bucle
5      DEC r16                    ; Decrementar
6      BRNE bucle_contador        ; Repetir si no es cero
7
8      ; Continúa aquí cuando r16 = 0
```

Control de desbordamiento:

```
1      ADD r16, r17                ; Operación que puede desbordar
2      BRVS error_overflow          ; Saltar si hay overflow
3      BRVC operacion_ok            ; Saltar si no hay overflow
4                                   ; (en complemento a 2)
```

Instrucciones SBRC y SBRS

Skip if Bit in Register is Cleared/Set

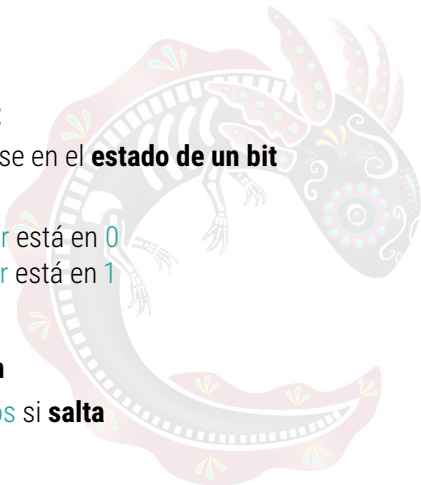
Saltan la **siguiente instrucción** basándose en el **estado de un bit** específico en un **registro**

SBRC Rr, b: Salta si el **bit b** del **registro Rr** está en **0**

SBRS Rr, b: Salta si el **bit b** del **registro Rr** está en **1**

Características:

- Solo saltan la **siguiente instrucción**
- Tiempo: **1 ciclo** si **no salta**, **2-3 ciclos** si **salta**
- Útiles para **polling de estados**



Instrucciones SBRC y SBRS

Ejemplos:

```
1 IN r16, PINB           ; Leer Puerto B
2 SBRC r16, 0             ; Bit 0 esta en 0?(boton presionado)
3 RJMP boton_no_presionado
4 ; Aqui se ejecuta si boton esta presionado
5
6 SBRS r16, 7             ; Bit 7 esta en 1?
7 RJMP bit7_bajo
8 ; Aqui se ejecuta si bit 7 esta alto
```

Instrucciones SBIC y SBIS

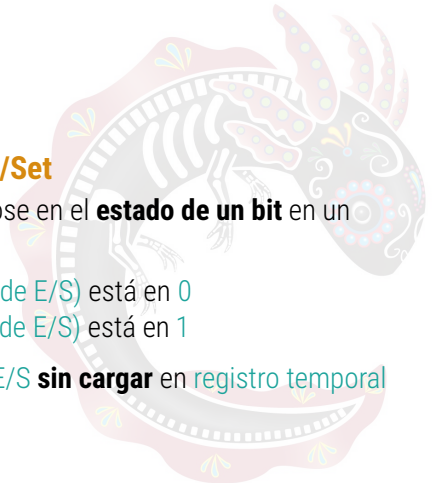
Skip if Bit in I/O Register is Cleared/Set

Saltan la **siguiente instrucción** basándose en el **estado de un bit** en un **registro de E/S**

SBIC P, b: Salta si el **bit b** del **registro P** (de E/S) está en 0

SBIS P, b: Salta si el **bit b** del **registro P** (de E/S) está en 1

Ventaja: **Acceso directo** a **registros de E/S sin cargar** en **registro temporal**



Instrucciones SBIC y SBIS

Ejemplos - Lectura de pulsadores:

```

1 ; Esperar hasta que se presione boton en PB0
2 esperar_boton:
3     SBIC PINB, 0           ; Boton presionado?(activo bajo)
4     RJMP esperar_boton    ; No, seguir esperando
5     ; Boton presionado, continuar
6
7     ; Verificar estado de LED
8     SBIS PORTB, 7         ; LED encendido?
9     RJMP encender_led     ; No, encenderlo
10    RJMP apagar_led       ; Si, apagarlo

```

Polling de estado de comunicación:

```

1 ; Esperar hasta que UART este listo para transmitir
2 esperar_uart:
3     SBIS UCSRA, UDRE      ; Buffer de transmision vacio?
4     RJMP esperar_uart     ; No, esperar
5     OUT UDR, r16          ; Si, transmitir dato

```

Ejemplo 1: Máquina de Estados

```

1 ; Estados: 0=IDLE, 1=RUNNING, 2=PAUSED, 3=ERROR
2 .def estado_actual = r16
3 .def evento = r17
4
5 maquina_estados:
6     ; Verificar rango de estado valido
7     CPI estado_actual, 4
8     BRSH estado_error      ; Si >= 4, error
9
10    ; Calcular direccion de estado
11    LDI ZL, low(tabla_estados*2)
12    LDI ZH, high(tabla_estados*2)
13    LSL estado_actual        ; estado * 2 (tamano de entrada)
14    ADD ZL, estado_actual
15    ADC ZH, ZH               ; Propagar carry
16
17    IJMP                    ; Saltar al manejador de estado
18
19 tabla_estados:
20    RJMP estado_idle        ; Estado 0

```

Ejemplo 1: Máquina de Estados

```
1      RJMP estado_running      ; Estado 1
2      RJMP estado_paused      ; Estado 2
3      RJMP estado_error       ; Estado 3
4
5 estado_idle:
6      CPI evento, 1            ; Evento START?
7      BRNE check_error_idle
8      LDI estado_actual, 1     ; Cambiar a RUNNING
9      RJMP maquina_estados
10
11 check_error_idle:
12      CPI evento, 3            ; Evento ERROR?
13      BRNE fin_idle
14      LDI estado_actual, 3     ; Cambiar a ERROR
15
16 fin_idle:
17      RET
18
19 estado_running:
20      CPI evento, 2            ; Evento PAUSE?
21      BRNE check_error_running
22      LDI estado_actual, 2     ; Cambiar a PAUSED
```

Ejemplo 1: Máquina de Estados

```
1      LDI estado_actual, 2      ; Cambiar a PAUSED
2      RJMP maquina_estados
3  check_error_running:
4      CPI evento, 3            ; Evento ERROR?
5      BRNE fin_running
6      LDI estado_actual, 3      ; Cambiar a ERROR
7  fin_running:
8      RET
```

Ejemplo 2: Bucles Anidados

```
1 ; Bucle doble: for(i=0; i<10; i++) for(j=0; j<5; j++)
2 .def i = r16
3 .def j = r17
4 .def temp = r18
5
6 bucles_anidados:
7     CLR i                ; i=0
8
9     bucle_externo:
10        CLR j             ; j=0 (inicializar bucle interno)
11
12    bucle_interno:
13        ; Código del bucle interno aquí
14        RCALL procesar_ij  ; Procesar con i, j actuales
15
16        INC j              ; j++
17        CPI j, 5           ; j < 5?
18        BRLO bucle_interno ; Si, continuar bucle interno
19
20    ; Bucle interno terminado
```

Ejemplo 2: Bucles Anidados

```
1      INC i                      ; i++
2      CPI i, 10                 ; i < 10?
3      BRLO bucle_externo        ; Si, continuar bucle externo
4
5      ; Ambos bucles terminados
6      RET
7
8  procesar_ij:
9      ; Ejemplo: calcular temp = i * 5 + j
10     MOV temp, i
11     LDI r19, 5
12     MUL temp, r19              ; i * 5
13     MOV temp, r0               ; Parte baja del resultado
14     ADD temp, j                ; + j
15
16     ; Usar temp para lo que sea necesario
17     OUT PORTB, temp           ; Mostrar resultado
18     RCALL delay_corto        ; Delay para visualizar
19     RET
```

Ejemplo 3: Control de LED con Botones

```

1 ; Sistema de control: 4 botones controlan 8 LEDs
2 ; Botones en PORTC (PC0-PC3), LEDs en PORTB (PB0-PB7)
3
4 .def botones = r16
5 .def leds = r17
6 .def patron = r18
7
8 main_control:
9 ; Configurar puertos
10     LDI r19, 0x00
11     OUT DDRC, r19           ; PORTC como entrada (botones)
12     LDI r19, 0xFF
13     OUT DDRC, r19           ; Activar pull-ups
14     OUT DDRB, r19           ; PORTB como salida (LEDs)
15
16     CLR leds                ; Inicializar LEDs apagados
17
18 bucle_principal:
19     IN botones, PINC         ; Leer botones
20     COM botones              ; Invertir (activo bajo)

```

Ejemplo 3: Control de LED con Botones

```
1  ANDI botones, 0x0F          ; Mascara para 4 botones
2
3  ; Decodificar botones
4  SBRC botones, 0             ; Boton 0 presionado?
5  RCALL boton_0
6
7  SBRC botones, 1             ; Boton 1 presionado?
8  RCALL boton_1
9
10 SBRC botones, 2             ; Boton 2 presionado?
11 RCALL boton_2
12
13 SBRC botones, 3             ; Boton 3 presionado?
14 RCALL boton_3
15
16 OUT PORTB, leds             ; Actualizar LEDs
17 RCALL delay_debounce        ; Anti-rebote
18 RJMP bucle_principal
19
20 boton_0:                    ; Encender todos los LEDs
```

Ejemplo 3: Control de LED con Botones

```

1      LDI leds, 0xFF
2      RET
3
4      boton_1:                                ; Apagar todos los LEDs
5          CLR leds
6          RET
7
8      boton_2:                                ; Patron alternado
9          LDI leds, 0b10101010
10         RET
11
12     boton_3:                                ; Rotar LEDs
13         LSL leds                            ; Desplazar izquierda
14         BRCC no_wrap                        ; Si no hay carry, no hay wrap
15         ORI leds, 0x01                     ; Establecer LSB
16     no_wrap:
17         TST leds                            ; Todos los LEDs apagados?
18         BRNE fin_boton_3
19         LDI leds, 0x01                     ; Reiniciar con 1 LED
20     fin_boton_3:
21         RET

```

Comparación de Rendimiento

Instrucción	Ciclos	Palabras	Rango	Uso Recomendado
RJMP	2	1	$\pm 2K$	Salto cercanos
JMP	3	2	4M	Salto lejanos
RCALL	3	1	$\pm 2K$	Subrutinas cercanas
CALL	4	2	4M	Subrutinas lejanas
I JMP	2	1	64K	Tablas de salto
ICALL	3	1	64K	Apuntadores a función
BR _{xx}	1/2	1	± 64	Decisiones simples
SBRC/SBRS	1/2/3	1	+1	Polling de bits
RET	4	1	-	Retorno de subrutina
RETI	4	1	-	Retorno de interrupción

Comparación de Rendimiento

Recomendaciones

- Usar **RJMP/RCALL** cuando el **destino** esté **dentro del rango**
- **IJMP/ICALL** son ideales para **máquinas de estado**
- Los **saltos condicionales** solo **avanzan**, **usar con cuidado**
- **SBRC/SBRS** son muy **eficientes** para **polling** de **un solo bit**

Mejores Prácticas

1. Estructura de programa eficiente:

```
1 ; Colocar codigo frecuentemente usado cerca del inicio
2 .org 0x0000
3     RJMP main                ; Vector de reset
4 .org 0x0002
5     RJMP ext_int0           ; Vector de interrupcion externa
6
7 main:
8     ; Inicializacion
9     RCALL init_ports
10    RCALL init_timers
11
12 ; Bucle principal (usar RJMP para eficiencia)
13 main_loop:
14     RCALL process_inputs
15     RCALL update_outputs
16     RJMP main_loop          ; Mas eficiente que JMP
```

Mejores Prácticas

2. Optimización de decisiones:

```
1 ; Malo - multiples saltos
2     CPI r16, 10
3     BREQ caso_10
4     RJMP siguiente_test
5 caso_10:
6     ; codigo
7     RJMP fin
8 siguiente_test:
9     ; mas tests...
10
11 ; Mejor - estructura mas directa
12     CPI r16, 10
13     BRNE test_20                ; Saltar si no es 10
14     ; codigo para caso 10
15     RJMP fin
16 test_20:
17     CPI r16, 20
18     BRNE test_30
```

Técnicas Avanzadas

1. Desenrollado de bucles (Loop Unrolling):

```
1 ; Bucle normal (mas ciclos de control)
2     LDI r17, 8
3 loop_normal:
4     LSL r16                ; Operacion
5     DEC r17
6     BRNE loop_normal
7
8 ; Bucle desenrollado (menos overhead)
9 LSL r16                ; 8 operaciones explicitas
10 LSL r16
11 LSL r16
12 LSL r16
13 LSL r16
14 LSL r16
15 LSL r16
16 LSL r16
```

Técnicas Avanzadas

2. Predicción de saltos:

```
1 ; Organizar codigo para que el caso mas comun no salte
2 SBIS PINB, 0 ; Verificar boton (caso comun:
    no presionado)
3 RJMP procesar_boton ; Saltar solo cuando se
    presiona
4 ; Codigo para caso comun (boton no presionado)
5 RJMP continuar
6
7 procesar_boton:
8     ; Codigo menos frecuente
9 continuar:
10    ; Continuar programa
```

Resumen - Saltos Incondicionales

Instrucción	Función	Rango	Uso Típico
JMP k	Salto absoluto	4M words	Saltos lejanos, reset
RJMP k	Salto relativo	$\pm 2K$ words	Bucles, saltos locales
I JMP	Salto indirecto	64K words	Tablas de salto, FSM
CALL k	Llamada absoluta	4M words	Subrutinas lejanas
RCALL k	Llamada relativa	$\pm 2K$ words	Subrutinas locales
ICALL	Llamada indirecta	64K words	Apuntadores a función
RET	Retorno subrutina	-	Fin de función
RETI	Retorno interrupción	-	Fin de ISR

Resumen - Saltos Condicionales

Por banderas SREG:

Condición	Instrucción
Igual	BREQ
No igual	BRNE
Carry set	BRCS/BRLO
Carry clear	BRCC/BRSH
Negativo	BRMI
Positivo	BRPL
Menor (signed)	BRLT
Mayor/igual (signed)	BRGE

Por estado de bits:

Condición	Instrucción
Bit registro = 0	SBRC
Bit registro = 1	SBRs
Bit I/O = 0	SBIC
Bit I/O = 1	SBIS
Registros iguales	CPSE

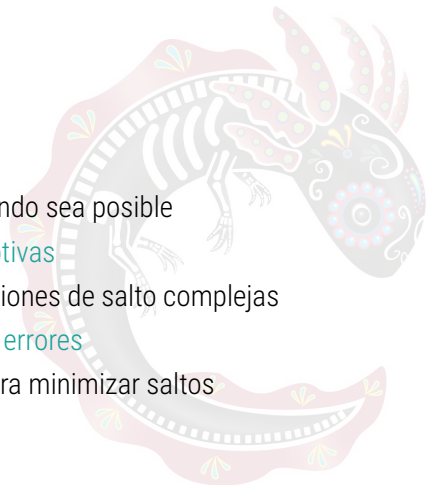
Características comunes:

- Rango limitado: ± 64 words (saltos condicionales), +1 instrucción (skip)
- Tiempo de ejecución: 1-3 ciclos según si salta o no
- Fundamentales para **estructuras de control**

Recomendaciones Finales

Mejores Prácticas

- 1 **Eficiencia:** Usar **RJMP/RCALL** cuando sea posible
- 2 **Legibilidad:** Usar **etiquetas descriptivas**
- 3 **Mantenimiento:** **Documentar** decisiones de salto complejas
- 4 **Robustez:** Implementar **manejo de errores**
- 5 **Optimización:** **Organizar código** para minimizar saltos



Recomendaciones Finales

Puntos Críticos

- Siempre **inicializar** el **Stack Pointer** antes de usar **CALL/RET**
- Los **saltos condicionales** tienen **rango limitado**
- **RETI** debe usarse solo en **rutinas de interrupción**
- **IJMP/ICALL** requieren **dirección en palabras** (no bytes)

Casos de Uso Comunes

- **Bucles: RJMP** con **contadores** y **BRNE**
- **Máquinas de estado: IJMP** con **tablas**
- **Polling: SBIC/SBIS** para **verificar hardware**
- **Menús: CALL/RET** para **modularidad**